HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant

High-Level Design Document

Submitted by: Bishwadeo

---

1. Introduction

This project presents the design of a **Retrieval-Augmented Generation (RAG)-based Customer Support Assistant**. The system is developed to answer user queries using a PDF knowledge base by combining information retrieval techniques with response generation.

The system improves traditional customer support by providing **context-aware responses** and supports **Human-in-the-Loop (HITL)** escalation for cases where the system is not confident.

---

2. Problem Statement

Traditional customer support systems face several limitations:

- Provide generic or incorrect responses
- Do not utilize internal knowledge bases effectively
- Cannot handle unknown or complex queries

There is a need for a system that:

- Uses a structured knowledge base (PDF)
- Retrieves relevant information efficiently
- Generates meaningful responses
- Escalates queries when necessary

---

3. System Overview

The system is designed to:

- Accept user queries
- Retrieve relevant document chunks using embeddings
- Generate answers based on retrieved context
- Use a workflow-based decision system
- Escalate to human agents when confidence is low

Key Features

- PDF-based knowledge processing
- Semantic search using embeddings
- Context-aware answer generation
- LangGraph-based workflow execution
- Conditional routing logic
- Human-in-the-Loop (HITL) support

---

4. Architecture Overview

The system follows a pipeline architecture integrated with a workflow engine.

Architecture Flow

User → Query Input → LangGraph Workflow → Retriever (ChromaDB) → Answer Generator
→ Decision Node → Output / HITL

---

5. Component Description

5.1 Document Loader

Loads the PDF file and extracts text using a document loader.

5.2 Chunking Module

Splits the document into smaller chunks using a text splitter.

- Chunk size: 500
- Overlap: 50

5.3 Embedding Model

Converts text chunks into vector representations using HuggingFace embeddings.

5.4 Vector Database (ChromaDB)

Stores embeddings and enables similarity-based retrieval.

5.5 Retriever

Retrieves top relevant chunks based on semantic similarity.

5.6 Answer Generator

Generates answers using retrieved document context.
(Currently implemented using rule-based logic; can be extended to LLM API)

5.7 LangGraph Workflow Engine

Controls execution flow using nodes and edges:

- Retrieve Node
- Generate Node
- Decision Node
- HITL Node
- Output Node

5.8 Routing Layer

Determines whether:

- Answer is returned
- Query is escalated

5.9 HITL Module

Handles human intervention when system confidence is low.

---

6. Data Flow

6.1 Document Processing Flow

1. PDF is loaded
2. Text is extracted
3. Text is split into chunks
4. Chunks are converted into embeddings
5. Embeddings are stored in ChromaDB

6.2 Query Processing Flow

1. User submits query
2. Query is passed to retriever
3. Relevant chunks are retrieved
4. Answer is generated
5. Confidence is evaluated
6. Decision is made:
    - High confidence → Output
    - Low confidence → HITL

---

7. Technology Stack

- Python
- LangChain
- ChromaDB
- LangGraph
- HuggingFace Embeddings

---

8. Scalability Considerations

- Efficient handling of large documents using chunking
- Vector-based retrieval for fast search
- Capability to handle multiple queries
- Potential integration with APIs for improved performance

---

9. Conclusion

The proposed system provides an efficient and scalable solution for customer support automation using RAG architecture and workflow-based decision-making. The integration of HITL ensures reliability and robustness in handling complex queries.